hw3

Yifeng Huang

This is the link to the repo: https://github.com/YifengHuang-Bob/Homework3

Problem 1

(a) Download and merge the datasets

```
library(haven)

aux <- read_xpt("data/AUX_I.xpt")
demo <- read_xpt("data/DEMO_I.xpt")

nhanes <- merge(aux, demo, by = "SEQN")

dim(nhanes)
```

```
[1] 4582  119
```

(b) Data cleaning

```
nhanes$RIAGENDR <- factor(
  nhanes$RIAGENDR,
  levels = c(1, 2),
  labels = c("Male", "Female")
)
```

```
nhanes$DMDCITZN[nhanes$DMDCITZN %in% c(7, 9)] <- NA

nhanes$DMDCITZN <- factor(
  nhanes$DMDCITZN,
  levels = c(1, 2),
  labels = c("Citizen", "Not a citizen")
)
```

```r
nhanes$INDHHIN2[nhanes$INDHHIN2 %in% c(77, 99)] <- NA

nhanes$INDHHIN2 <- factor(
  nhanes$INDHHIN2,
  levels = c(1:10, 14, 15, 13, 12),
  labels = c(
    "$0-$4,999",
    "$5,000-$9,999",
    "$10,000-$14,999",
    "$15,000-$19,999",
    "$20,000-$24,999",
    "$25,000-$34,999",
    "$35,000-$44,999",
    "$45,000-$54,999",
    "$55,000-$64,999",
    "$65,000-$74,999",
    "$75,000-$99,999",
    "$100,000 and over",
    "Under $20,000 (broad)",
    "$20,000 and over (broad)"
  )
)
```

The original income codes do not follow a strictly increasing income order. I rearranged the detailed income categories from lowest to highest and placed the two broad, overlapping categories separately at the end, as they cannot be precisely ordered among the detailed income categories.

(c) Poisson regression models

```r
nhanes$income_numeric <- as.numeric(nhanes$INDHHIN2)

nhanes$income_numeric[nhanes$income_numeric > 12] <- NA
```

```r
model_1r <- glm(
  AUXTWIDR ~ RIAGENDR,
  data = nhanes,
  family = poisson(link = "log")
)
```

```r
model_2r <- glm(
  AUXTWIDR ~ RIAGENDR + DMDCITZN + DMDHHSZA + income_numeric,
```

```
  data = nhanes,
  family = poisson(link = "log")
)
```

```
model_1l <- glm(
  AUXTWIDL ~ RIAGENDR,
  data = nhanes,
  family = poisson(link = "log")
)
```

```
model_2l <- glm(
  AUXTWIDL ~ RIAGENDR + DMDCITZN + DMDHHSZA + income_numeric,
  data = nhanes,
  family = poisson(link = "log")
)
```

```
irr_table <- data.frame(
  Predictor = c(
    "Intercept (exp(coefficient))",
    "Female",
    "Not a citizen",
    "Children (per unit)",
    "Income level (per unit)"
  ),
  `1R` = c(exp(coef(model_1r)), rep(NA_real_, 3)),
  `2R` = exp(coef(model_2r)),
  `1L` = c(exp(coef(model_1l)), rep(NA_real_, 3)),
  `2L` = exp(coef(model_2l)),
  check.names = FALSE
)

options(knitr.kable.NA = "-")
```

```
knitr::kable(
  irr_table,
  row.names = FALSE,
  digits = 4,
  caption = "Exponentiated coefficients from the four Poisson regression models"
)
```

Predictor	1R	2R	1L	2L
Intercept (exp(coefficient))	84.3174	88.4374	84.7584	88.8213
Female	1.0093	1.0150	1.0130	1.0187
Not a citizen	-	1.0469	-	1.0218
Children (per unit)	-	0.9944	-	0.9830
Income level (per unit)	-	0.9921	-	0.9935

```
model_stats <- data.frame(
  Statistic = c("Sample size", "Pseudo-R2 (deviance-based)", "AIC"),
  `1R` = c(
    format(nobs(model_1r), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_1r) / model_1r$null.deviance),
    format(round(AIC(model_1r)), big.mark = ",")
  ),
  `2R` = c(
    format(nobs(model_2r), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_2r) / model_2r$null.deviance),
    format(round(AIC(model_2r)), big.mark = ",")
  ),
  `1L` = c(
    format(nobs(model_1l), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_1l) / model_1l$null.deviance),
    format(round(AIC(model_1l)), big.mark = ",")
  ),
  `2L` = c(
    format(nobs(model_2l), big.mark = ","),
    sprintf("%.6f", 1 - deviance(model_2l) / model_2l$null.deviance),
    format(round(AIC(model_2l)), big.mark = ",")
  ),
  check.names = FALSE
)

knitr::kable(
  model_stats,
  row.names = FALSE,
  caption = "Model statistics for the four Poisson regression models"
)
```

Statistic	1R	2R	1L	2L
Sample size	4,149	3,705	4,103	3,665
Pseudo-R2 (deviance-based)	0.000106	0.005717	0.000200	0.003631
AIC	96,618	84,916	98,685	86,598

(d) Hypothesis tests

```
gender_test <- coef(summary(model_2l))["RIAGENDRFemale", ]

gender_test
```

```
    Estimate   Std. Error      z value     Pr(>|z|)
1.857273e-02 3.599573e-03 5.159705e+00 2.473393e-07
```

```
exp(coef(model_2l)["RIAGENDRFemale"])
```

```
RIAGENDRFemale
      1.018746
```

```
emm_gender <- emmeans::emmeans(
  model_2l,
  ~ RIAGENDR
)

summary(emm_gender, type = "response")
```

```
 RIAGENDR rate    SE  df asymp.LCL asymp.UCL
 Male     84.8 0.252 Inf      84.3      85.3
 Female   86.3 0.249 Inf      85.9      86.8

Results are averaged over the levels of: DMDCITZN
Confidence level used: 0.95
Intervals are back-transformed from the log scale
```

```
gender_diff <- pairs(
  emmeans::regrid(emm_gender, transform = "response"),
  reverse = TRUE
```

```
)

summary(gender_diff, infer = c(TRUE, TRUE))
```

```
 contrast      estimate    SE  df asymp.LCL asymp.UCL z.ratio p.value
 Female - Male     1.59 0.308 Inf     0.986      2.19   5.161 <0.0001

Results are averaged over the levels of: DMDCITZN
Confidence level used: 0.95
```

Test 1 — IRR Comparison

For Model 2L, the estimated female-to-male IRR is 1.0187. The Wald test gives $z = 5.160$ and $p = 2.47 \times 10^{-7}$. Therefore, we reject the null hypothesis that the IRR equals 1 at the 5% significance level. Holding other covariates constant, females have approximately 1.87% higher expected left-ear tympanometric width than males.

Test 2 — Predicted Value Comparison

The adjusted predicted values are 84.8 for males and 86.3 for females. The estimated difference (Female – Male) is 1.59, with a 95% confidence interval of $[0.986, 2.19]$. The test gives $z = 5.161$ and $p < 0.0001$. Since the confidence interval excludes 0, we reject the null hypothesis of equal predicted values at the 5% significance level.

Note on Overdispersion

These results are based on the standard Poisson model. Since the model shows evidence of overdispersion, the reported p-values and confidence intervals should be interpreted cautiously.

Problem 2 - Sakila

```
library(DBI)

con <- dbConnect(
  RSQLite::SQLite(),
  "data/sakila_master.db"
)
```

(a) Customers and active percentages by store

Method 1 - SQL + R

```
customer_data <- dbGetQuery(
  con,
  "SELECT store_id, active FROM customer"
)
```

```r
customer_counts <- table(customer_data$store_id)

active_percent <- tapply(
  customer_data$active == "1",
  customer_data$store_id,
  mean
) * 100

result_r <- data.frame(
  store_id = as.integer(names(customer_counts)),
  customers = as.integer(customer_counts),
  percent_active = as.numeric(active_percent)
)

knitr::kable(
  result_r,
  digits = 2,
  row.names = FALSE,
  caption = "Customer counts and active percentages by store (SQL + R)"
)
```

store_id	customers	percent_active
1	326	97.55
2	273	97.44

Method 2 - Single SQL Query

```r
result_sql <- dbGetQuery(
  con,
  "SELECT
     store_id,
     COUNT(*) AS customers,
     ROUND(
       100.0 * SUM(CASE WHEN active = '1' THEN 1 ELSE 0 END) / COUNT(*),
       2
     ) AS percent_active
   FROM customer
   GROUP BY store_id
   ORDER BY store_id"
)
```

```
knitr::kable(
  result_sql,
  digits = 2,
  row.names = FALSE,
  caption = "Customer counts and active percentages by store (SQL only)"
)
```

store_id	customers	percent_active
1	326	97.55
2	273	97.44

Performance comparison

```
bench_a <- microbenchmark::microbenchmark(
  SQL_R = {
    data_r <- dbGetQuery(
      con,
      "SELECT store_id, active FROM customer"
    )

    counts_r <- table(data_r$store_id)

    percent_r <- tapply(
      data_r$active == "1",
      data_r$store_id,
      mean
    ) * 100

    data.frame(
      store_id = as.integer(names(counts_r)),
      customers = as.integer(counts_r),
      percent_active = as.numeric(percent_r)
    )
  },
  SQL_only = {
    dbGetQuery(
      con,
      "SELECT
         store_id,
         COUNT(*) AS customers,
         ROUND(
           100.0 * SUM(CASE WHEN active = '1' THEN 1 ELSE 0 END) / COUNT(*),
```

```
          2
        ) AS percent_active
      FROM customer
      GROUP BY store_id
      ORDER BY store_id"
    )
  },
  times = 100L
)

bench_summary <- summary(bench_a, unit = "us")

median_sql_r <- bench_summary$median[bench_summary$expr == "SQL_R"]
median_sql_only <- bench_summary$median[bench_summary$expr == "SQL_only"]

bench_summary
```

```
      expr     min      lq     mean   median       uq     max neval
1    SQL_R 498.700 548.901 601.1130 572.2510 623.8000 965.901   100
2 SQL_only 204.001 249.201 279.3121 267.2005 294.0515 701.801   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 267.20 and 572.25 microseconds, respectively. Thus, the SQL-only approach was about 2.14 times faster in this benchmark, likely because the aggregation was performed directly in the database.

(b) Staff names and countries

Method 1 - SQL + R

```
staff_data <- dbGetQuery(
  con,
  "SELECT staff_id, first_name, last_name, address_id FROM staff"
)

address_data <- dbGetQuery(
  con,
  "SELECT address_id, city_id FROM address"
)

city_data <- dbGetQuery(
  con,
```

```
  "SELECT city_id, country_id FROM city"
)

country_data <- dbGetQuery(
  con,
  "SELECT country_id, country FROM country"
)

staff_info <- merge(staff_data, address_data, by = "address_id")
staff_info <- merge(staff_info, city_data, by = "city_id")
staff_info <- merge(staff_info, country_data, by = "country_id")

result_r_b <- data.frame(
  staff_id = staff_info$staff_id,
  staff_name = paste(staff_info$first_name, staff_info$last_name),
  country = staff_info$country
)

result_r_b <- result_r_b[order(result_r_b$staff_id), ]

knitr::kable(
  result_r_b,
  row.names = FALSE,
  caption = "Staff names and countries (SQL + R)"
)
```

staff_id	staff_name	country
1	Mike Hillyer	Canada
2	Jon Stephens	Australia

Method 2 - Single SQL Query

```
result_sql_b <- dbGetQuery(
  con,
  "SELECT
     s.staff_id,
     s.first_name || ' ' || s.last_name AS staff_name,
     co.country
   FROM staff AS s
   JOIN address AS a ON s.address_id = a.address_id
   JOIN city AS ci ON a.city_id = ci.city_id
```

```
   JOIN country AS co ON ci.country_id = co.country_id
   ORDER BY s.staff_id"
)

knitr::kable(
  result_sql_b,
  row.names = FALSE,
  caption = "Staff names and countries (SQL only)"
)
```

staff_id	staff_name	country
1	Mike Hillyer	Canada
2	Jon Stephens	Australia

Performance comparison

```
bench_b <- microbenchmark::microbenchmark(
  SQL_R = {
    staff_b <- dbGetQuery(
      con,
      "SELECT staff_id, first_name, last_name, address_id FROM staff"
    )

    address_b <- dbGetQuery(
      con,
      "SELECT address_id, city_id FROM address"
    )

    city_b <- dbGetQuery(
      con,
      "SELECT city_id, country_id FROM city"
    )

    country_b <- dbGetQuery(
      con,
      "SELECT country_id, country FROM country"
    )

    info_b <- merge(staff_b, address_b, by = "address_id")
    info_b <- merge(info_b, city_b, by = "city_id")
    info_b <- merge(info_b, country_b, by = "country_id")
```

```
    result_b <- data.frame(
      staff_id = info_b$staff_id,
      staff_name = paste(info_b$first_name, info_b$last_name),
      country = info_b$country
    )

    result_b[order(result_b$staff_id), ]
  },
  SQL_only = {
    dbGetQuery(
      con,
      "SELECT
         s.staff_id,
         s.first_name || ' ' || s.last_name AS staff_name,
         co.country
       FROM staff AS s
       JOIN address AS a ON s.address_id = a.address_id
       JOIN city AS ci ON a.city_id = ci.city_id
       JOIN country AS co ON ci.country_id = co.country_id
       ORDER BY s.staff_id"
    )
  },
  times = 100L
)

bench_summary_b <- summary(bench_b, unit = "us")

median_sql_r_b <- bench_summary_b$median[bench_summary_b$expr == "SQL_R"]
median_sql_only_b <- bench_summary_b$median[bench_summary_b$expr == "SQL_only"]

bench_summary_b
```

```
      expr      min       lq      mean   median       uq      max neval
1    SQL_R 1538.001 1611.851 1779.2399 1658.651 1740.151 4548.200   100
2 SQL_only  156.100  181.251  231.0249  221.451  242.251  676.901   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 221.45 and 1658.65 microseconds, respectively. Thus, the SQL-only approach was about 7.49 times faster in this benchmark, likely because it performs the joins directly in the database, avoiding multiple data transfers and additional merging operations in R.

(c) Films rented for the highest dollar value

Method 1 - SQL + R

```
payment_data <- dbGetQuery(
  con,
  "SELECT rental_id, amount FROM payment"
)

rental_data <- dbGetQuery(
  con,
  "SELECT rental_id, inventory_id FROM rental"
)

inventory_data <- dbGetQuery(
  con,
  "SELECT inventory_id, film_id FROM inventory"
)

film_data <- dbGetQuery(
  con,
  "SELECT film_id, title FROM film"
)

film_payments <- merge(payment_data, rental_data, by = "rental_id")
film_payments <- merge(film_payments, inventory_data, by = "inventory_id")
film_payments <- merge(film_payments, film_data, by = "film_id")

max_amount <- max(film_payments$amount)

result_r_c <- unique(
  film_payments[film_payments$amount == max_amount, c("title", "amount")]
)

result_r_c <- result_r_c[order(result_r_c$title), ]

names(result_r_c) <- c("film_title", "payment_usd")
rownames(result_r_c) <- NULL

knitr::kable(
  result_r_c,
  digits = 2,
  row.names = FALSE,
```

```
  caption = "Films rented for the highest dollar value (SQL + R)"
)
```

film_title	payment_usd
FLINTSTONES HAPPINESS	11.99
MIDSUMMER GROUNDHOG	11.99
MINE TITANS	11.99
SCORPION APOLLO	11.99
SHOW LORD	11.99
STING PERSONAL	11.99
TIES HUNGER	11.99
TRAP GUYS	11.99
VIRTUAL SPOILERS	11.99

Method 2 - Single SQL Query

```
result_sql_c <- dbGetQuery(
  con,
  "SELECT DISTINCT
     f.title AS film_title,
     p.amount AS payment_usd
   FROM payment AS p
   JOIN rental AS r ON p.rental_id = r.rental_id
   JOIN inventory AS i ON r.inventory_id = i.inventory_id
   JOIN film AS f ON i.film_id = f.film_id
   WHERE p.amount = (SELECT MAX(amount) FROM payment)
   ORDER BY f.title"
)

knitr::kable(
  result_sql_c,
  digits = 2,
  row.names = FALSE,
  caption = "Films rented for the highest dollar value (SQL only)"
)
```

film_title	payment_usd
FLINTSTONES HAPPINESS	11.99
MIDSUMMER GROUNDHOG	11.99
MINE TITANS	11.99
SCORPION APOLLO	11.99
SHOW LORD	11.99
STING PERSONAL	11.99
TIES HUNGER	11.99
TRAP GUYS	11.99
VIRTUAL SPOILERS	11.99

Performance comparison

```
bench_c <- microbenchmark::microbenchmark(
  SQL_R = {
    payment_c <- dbGetQuery(
      con,
      "SELECT rental_id, amount FROM payment"
    )

    rental_c <- dbGetQuery(
      con,
      "SELECT rental_id, inventory_id FROM rental"
    )

    inventory_c <- dbGetQuery(
      con,
      "SELECT inventory_id, film_id FROM inventory"
    )

    film_c <- dbGetQuery(
      con,
      "SELECT film_id, title FROM film"
    )

    film_payments_c <- merge(payment_c, rental_c, by = "rental_id")
    film_payments_c <- merge(film_payments_c, inventory_c, by = "inventory_id")
    film_payments_c <- merge(film_payments_c, film_c, by = "film_id")
```

```
    max_amount_c <- max(film_payments_c$amount)

    result_c <- unique(
          film_payments_c[film_payments_c$amount  ==  max_amount_c,  c("title",
"amount")]
    )

    result_c <- result_c[order(result_c$title), ]

    names(result_c) <- c("film_title", "payment_usd")
    rownames(result_c) <- NULL

    result_c
  },
  SQL_only = {
    dbGetQuery(
      con,
      "SELECT DISTINCT
         f.title AS film_title,
         p.amount AS payment_usd
       FROM payment AS p
       JOIN rental AS r ON p.rental_id = r.rental_id
       JOIN inventory AS i ON r.inventory_id = i.inventory_id
       JOIN film AS f ON i.film_id = f.film_id
       WHERE p.amount = (SELECT MAX(amount) FROM payment)
       ORDER BY f.title"
    )
  },
  times = 100L
)

bench_summary_c <- summary(bench_c, unit = "us")

median_sql_r_c <- bench_summary_c$median[bench_summary_c$expr == "SQL_R"]
median_sql_only_c <- bench_summary_c$median[bench_summary_c$expr == "SQL_only"]

bench_summary_c
```

```
      expr       min        lq      mean    median        uq        max neval
1    SQL_R 31670.102 34738.652 38303.259 36766.651 39621.500 117624.601   100
2 SQL_only   806.701  1058.401  2453.631  3294.001  3535.351   5045.501   100
```

The microbenchmark results show that the SQL-only approach is faster than the SQL + R approach. Over 100 evaluations, the median execution times were approximately 3294.00 and 36766.65 microseconds, respectively. Thus, the SQL-only approach was about 11.16 times faster in this benchmark, likely because it performs the joins and filtering directly in the database, reducing data transfer and additional processing in R.